

Módulo 2 — Árboles, bosques y boosting

Curso de Datos Móviles y Machine Learning para la Estimación de la Pobreza

Bloque teórico — Día 2

16 de junio de 2026

Banco Mundial - INEC - USFQ

Agenda del bloque

1. Introducción: del problema de pobreza a los árboles
2. Árboles CART y poda
3. Random Forest, OOB e importancia
4. Boosting (potenciación por gradiente)
5. XGBoost y otras implementaciones
6. Flujo de trabajo: tuning y evaluación
7. Mapa comparativo y Q&A

Al final del módulo el participante podrá:

1. Explicar cómo un árbol parte los datos y por qué hay que podarlo
2. Entender por qué Random Forest reduce la varianza
3. Interpretar importancias de variables *con sus límites*
4. Explicar boosting como corrección secuencial de errores
5. Distinguir entre Random Forest y boosting, y cuándo usar cada uno
6. Elegir una estrategia de búsqueda de hiperparámetros con criterio

Introducción: del problema a los árboles

El problema concreto

Queremos **predecir un resultado** a partir de variables observables:

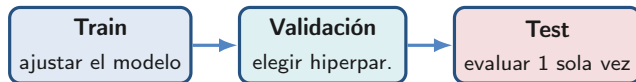
- ¿Es pobre este hogar? (clasificación: pobre / no pobre)
- ¿Cuál es su gasto per cápita? (regresión: un número)

A partir de variables x : ingreso, educación, tamaño del hogar, acceso a servicios, indicadores de telefonía móvil, etc.

Nota

Buscamos una función $\hat{f}(x)$ que, dado lo que sabemos del hogar, devuelva una predicción *útil* para hogares que el modelo nunca vio.

Recordatorio: ¿cómo se evalúa un modelo? (Módulo 1)



- El **test** se reserva hasta el final: mide generalización honesta
- La validación cruzada (sobre train) sirve para *elegir* el modelo

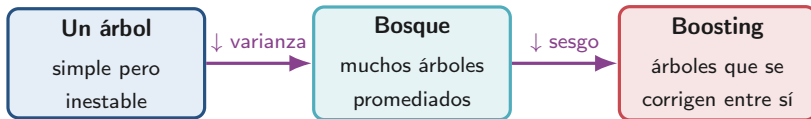
¿Por qué árboles?

Pensamos en decisiones encadenadas, igual que un humano:

“Si el ingreso está bajo la línea y la educación es baja $\Rightarrow$ probablemente pobre.”

- Capturan **umbrales naturales** (línea de pobreza, edades de corte)
- No exigen escalar ni transformar variables
- Mezclan variables continuas y categóricas sin esfuerzo
- Detectan **interacciones** solas (educación importa solo si el ingreso es bajo)

El recorrido del módulo en un diagrama



- Empezamos con la pieza básica y vamos sumando “potencia”
- Cada paso resuelve una debilidad del anterior

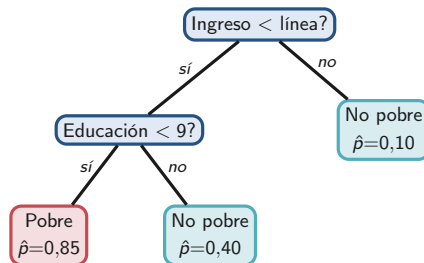
Classification and Regression Tree

CART

La idea: una secuencia de preguntas binarias

- Árbol = preguntas “sí/no” anidadas
- En cada **hoja**, una predicción constante
- No paramétrico y adaptativo

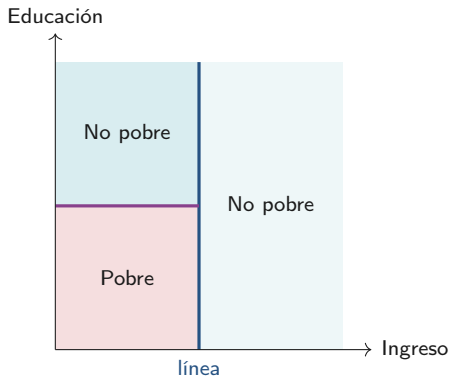
- Primera nivel: nodo raíz
- Otros niveles: nodos
- Nivel final: hojas



Un árbol es una partición del espacio

El mismo árbol divide el plano en **rectángulos**; cada hoja es una región.

- Fronteras siempre paralelas a los ejes
- Predicción constante dentro de cada región



¿Por dónde corta? Ganancia de pureza

Probamos cada variable y cada umbral, y elegimos el corte que deja los dos lados lo más “puros” posible (hogares parecidos juntos).

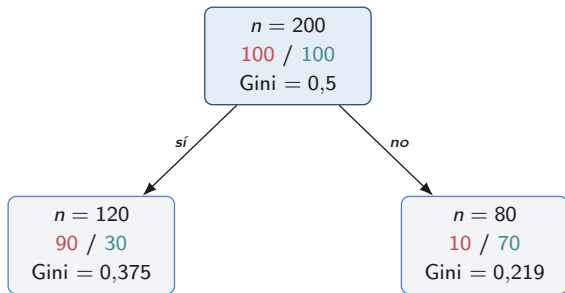
$$\Delta i(t) = \underbrace{i(t)}_{\text{impureza antes}} - \underbrace{\frac{n_L}{n_t} i(t_L) - \frac{n_R}{n_t} i(t_R)}_{\text{impureza después (ponderada)}}$$

- Búsqueda **greedy**: el mejor corte *local*, sin mirar el futuro
- Nunca se vuelve atrás (sin backtracking)

Ejemplo: ¿cuánta pureza gana un corte?

Corte candidato: ingreso < línea de pobreza

■ pobres ■ no pobres



Recordatorio (Gini): $i_G = 1 - \sum_k \hat{p}_k^2$

$$\begin{aligned}\Delta i &= 0,5 - \left[\frac{120}{200}(0,375) + \frac{80}{200}(0,219) \right] \\ &= 0,5 - 0,313 \approx 0,19\end{aligned}$$

Nota

La mezcla 50/50 del padre se separa: un lado queda 75 % pobre y el otro 88 % no pobre. Baja la impureza $\Rightarrow$ ganamos pureza.

¿Cómo se mide la “pureza”?

Regresión: qué tan dispersos están los valores (varianza)

$$i(t) = \frac{1}{n_t} \sum_{i \in t} (y_i - \bar{y}_t)^2$$

Clasificación: qué tan mezcladas están las clases

$$i_G(t) = 1 - \sum_k \hat{p}_k^2 \quad (\text{Gini})$$

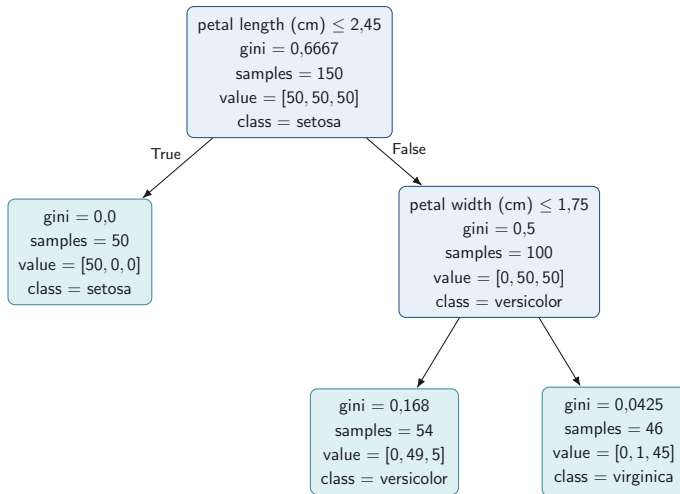
$$i_H(t) = - \sum_k \hat{p}_k \log \hat{p}_k \quad (\text{Entropía})$$

Nota

Un nodo con 100 % pobres tiene impureza 0; uno mitad y mitad, la máxima. Gini y entropía dan resultados casi iguales en la práctica.

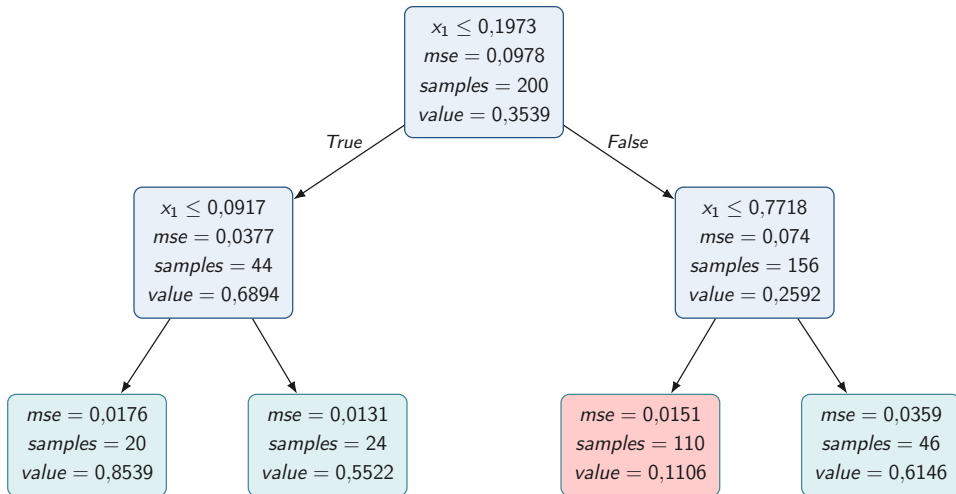
Árbol de clasificación: Gini, conteos y clase

Tarea: Predecir la especie de planta por su tamaño de pétalos (cm)



Árbol de regresión: predicción y error por hoja

Tarea: Predecir y (*value*) cuando $x_1 = 0,6$



Propiedades útiles del árbol

- **Invariante** a transformaciones monótonas de las variables
- No requiere escalar ni normalizar
- Maneja categóricas y continuas en el mismo modelo
- Captura **interacciones** automáticamente al anidar cortes

Nota

Menos preprocesamiento $\Rightarrow$ menos oportunidades de cometer errores de fuga de información (*leakage*) en el flujo.

El peligro: sobreajuste

- Árbol completo → una hoja por hogar → **memoriza**
- Alta varianza: cambia un poco los datos y sale otro árbol
- Solución: **podar** o limitar el crecimiento

Nota

Un árbol enorme acierta todo en train, pero falla en hogares nuevos. El error de validación tiene forma de U: baja, toca un mínimo y vuelve a subir.

puntaje del árbol = que tan mal predice + α (cuantas hojas tiene)

$$C_{\alpha}(T) = \underbrace{\sum_m \sum_{i \in R_m} L(y_i, \hat{y}_{R_m})}_{\text{error en train}} + \alpha \underbrace{|T|}_{\text{n. de hojas}}$$

Nota

Penalizamos árboles grandes. α pone precio a cada hoja: a mayor precio, árbol más chico.

- $\alpha \rightarrow 0$: árbol grande (sobreajusta)
- $\alpha \rightarrow \infty$: árbol pequeño (subajusta)
- α se elige por validación cruzada

- Alta **varianza** (inestable)
- Fronteras escalonadas; no capta tendencias suaves
- Sesgo hacia variables categóricas con muchos niveles

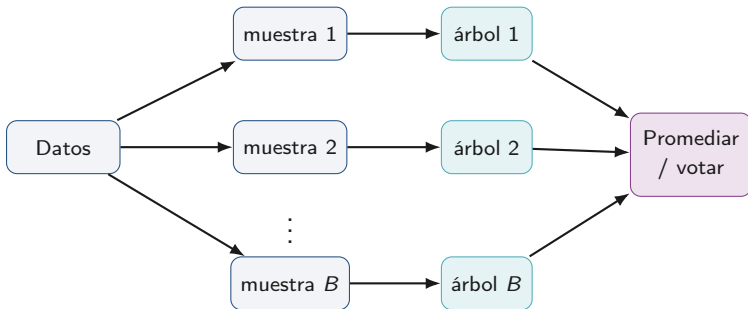
La idea que sigue

Si un árbol es inestable... **¿y si combinamos muchos?** Eso son los *ensambles*: bagging, Random Forest y boosting.

Random Forest

De un árbol a un bosque: bagging

Idea: entrenar muchos árboles sobre **muestras distintas** de los datos y luego **promediar** sus predicciones.



Las muestras se generan por *bootstrap*: al azar y **con reemplazo**.

¿Por qué ayuda promediar?

Nota

Si promedias muchos árboles, sus errores aleatorios (que apuntan en direcciones distintas) se **cancelan** entre sí, y queda la señal real.

- Promediar baja la varianza... **si** los árboles son distintos entre sí
- Árboles muy parecidos $\Rightarrow$ el promedio casi no mejora nada

Por eso el truco de Random Forest será forzar que los árboles se parezcan poco entre sí.

Random Forest: la receta

1. Tomar B muestras bootstrap de los datos
2. En **cada corte**, considerar solo m (variables seleccionadas al azar) de las p variables.
3. Construir cada árbol completo, sin podar
4. Agregar: promedio (regresión) o voto (clasificación)

Valor típico: $m \approx \sqrt{p}$ (clasificación) o $m \approx p/3$ (regresión).

Nota

El paso 2 es la clave: al ocultar variables en cada corte, obliga a los árboles a ser **distintos** $\Rightarrow$ el promedio baja más la varianza.

Hiperparámetros de Random Forest

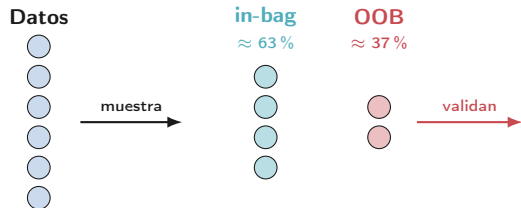
Hiperparámetro	Al aumentarlo...	Rango típico
variables por corte (m)	árboles más parecidos	$\sqrt{p}$ o $p/3$
n. de árboles (B)	más estable (sin sobreajustar)	500–2000
tamaño mínimo de hoja	árboles más superficiales	5–20

Nota

Agregar árboles *nunca* sobreajusta: solo estabiliza. El ajuste real está en m y en el tamaño de hoja.

OOB: validación “gratis”

- Cada árbol usa $\approx 63\%$ de los datos (su muestra bootstrap)
- El $\approx 37\%$ restante **no lo vio** (fuera de bolsa, OOB)
- Esos datos no vistos sirven para validarlo



Nota

Cada hogar es “dato de prueba” para todos los árboles que no lo vieron. Validamos **sin reservar datos aparte**.

- En Random Forest, el OOB puede reemplazar a la validación cruzada para ajustar hiperparámetros (más rápido)
- Útil sobre todo en datasets grandes

Salvedad importante

Para *comparar* Random Forest con otro modelo (p.ej. boosting), usar el **mismo** esquema de validación para todos (normalmente validación cruzada) y no mezclar.

Boosting (potenciación por gradiente)

La idea: corregir errores en cadena



- Cada árbol nuevo se enfoca en lo que el anterior **falló**
- Se suman con un **peso pequeño** ν (tasa de aprendizaje)

Random Forest: expertos independientes que votan. **Boosting:** una cadena de revisores donde cada uno arregla lo que falló el anterior.

¿Qué corrige cada árbol nuevo?

Cada árbol nuevo se entrena sobre los **errores que sobran** del modelo anterior, no sobre el objetivo original.

- En el caso más simple, ese “error a corregir” es el **residuo**:

$$\text{residuo} = \text{valor real} - \text{valor predicho}$$

- En general se usa el **gradiente** de la pérdida (de ahí “*gradient boosting*”): la misma receta sirve para clasificación, regresión, etc.

El residuo clásico es solo el caso particular más simple.

Cómo evitar el sobreajuste en boosting

- Tasa de aprendizaje pequeña ν (entre 0.01 y 0.1)
- Árboles poco profundos (profundidad 3–8)
- Submuestreo de filas en cada paso
- Parada temprana: detener cuando la validación deja de mejorar

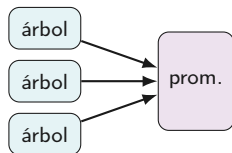
Regla práctica: ν pequeño + muchos árboles $>$ ν grande + pocos árboles.

Nota

A diferencia del bosque, aquí *sí* se puede poner demasiados árboles: por eso la parada temprana.

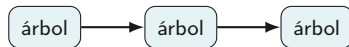
Random Forest vs. Boosting de un vistazo

Random Forest (paralelo)



reduce **varianza**

Boosting (secuencial)



reduce **sesgo**

	Random Forest	Boosting
Sensibilidad a hiperpar.	Baja	Alta
Costo computacional	Medio (paralelizable)	Alto (secuencial)

¿Cuándo preferir Random Forest?

- Datasets pequeños (RF es más estable)
- Poco tiempo para ajustar hiperparámetros
- Se quiere usar OOB sin validación cruzada externa
- Cuando el mantenimiento importa más que el último 1 % de desempeño

Nota

Muy a menudo RF es la opción correcta por gobernanza y simplicidad, aunque boosting dé un poco más de precisión.

XGBoost y otras implementaciones

El Gradient Boosting Machine (GBM) que usamos: XGBoost

- “Gradient boosting” es la *idea general*
- **XGBoost** es la *implementación* más usada (el “GBM” del flujo práctico)
- Qué añade XGBoost:
 - **Regularización explícita**: penaliza árboles complejos
 - **Poda incorporada**: no hace un corte si no mejora lo suficiente
 - Optimizado para correr **rápido**

Misma mecánica de boosting, pero con frenos de complejidad y muy optimizado.

Ventajas prácticas de XGBoost (útiles en encuestas)

- Manejo nativo de **valores faltantes** (aprende a dónde mandarlos): No requiere imputar valores, sabe por qué 'rama' enviarlos.
- **Pesos por observación**: Permite que unas observaciones valgan más que otras.
- Muy rápido en datos grandes (cortes basados en histogramas): agrupa los valores por bins y prueba divisiones en estos grupos.
- Permite imponer relaciones **monótonas** si se conocen: Definir relaciones que son conocidas. Es más coherente con la teoría con menos relaciones absurdas aprendidas.

Otras implementaciones (alternativas)

LightGBM más rápido y eficiente en datasets muy grandes

CatBoost mejor manejo nativo de variables categóricas

Hiperparámetros críticos en boosting

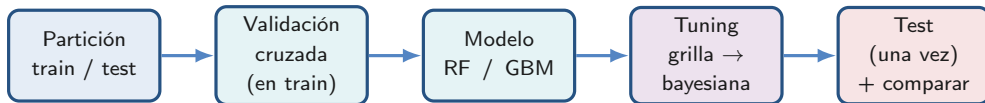
Hiperparámetro	Rango típico	Efecto
tasa de aprendizaje (ν)	0.01–0.1	↓: más estable, más árboles
profundidad máxima	3–8	↑: más complejo
n. de árboles	100–2000	usar con parada temprana
mínimo por hoja	1–10	controla hojas pequeñas
submuestreo	0.5–1.0	aleatoriza filas
regularización	0–10	penaliza complejidad

Nota

Buen punto de partida: $\nu = 0,05$, profundidad 6, y parada temprana.

Flujo de trabajo: tuning y evaluación

El flujo de trabajo del módulo



- Los hiperparámetros se eligen con validación cruzada *sobre train*
- El **test** se usa una sola vez, al final

El problema: muchos hiperparámetros, poco tiempo

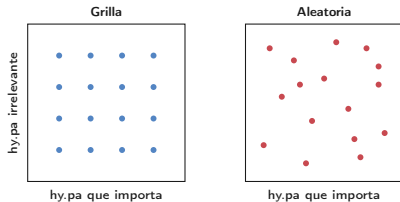
Cada combinación de hiperparámetros es un modelo distinto. Hay que probar varias y quedarse con la mejor *en validación*.

- Cada candidato se evalúa por validación cruzada
- El espacio crece rápido: 5 hiperparámetros con 4 valores → miles de modelos
- Pregunta clave: ¿cómo explorar el espacio con criterio?

Búsqueda en grilla vs. aleatoria

Grilla todas las combinaciones de una rejilla fija (auditable; crece rápido)

Aleatoria combinaciones al azar del espacio



Hallazgo: si pocos hiperparámetros importan, la aleatoria llega antes a una buena solución. La grilla afina valores que quizá no importan; la aleatoria prueba más valores *distintos* del que sí importa.

Búsqueda bayesiana: aprender sobre la marcha

En vez de probar a ciegas, usa lo **ya probado** para decidir con inteligencia qué combinación probar después.

1. Parte de combinaciones ya evaluadas (p.ej. las de la grilla)
2. Aprende qué zonas del espacio parecen **prometedoras**
3. Propone nuevas combinaciones y repite

Diferencia clave: la grilla evalúa una lista cerrada; la bayesiana **aprende y propone**. Suele necesitar menos evaluaciones.

Clasificación F1 (balance precisión/recall), ROC AUC, PR-AUC (mejor bajo desbalance)

Regresión RMSE (penaliza errores grandes), MAE (robusta a atípicos)

Comparar todos los modelos con la MISMA métrica y el MISMO esquema de validación.

Evaluación final y comparación

- Una vez elegidos los hiperparámetros con validación cruzada, entrenar en **todo train** y evaluar **una sola vez** en test
- Recién ahí se reportan las métricas finales

La comparación que importa

No es solo *quién gana*, sino si la mejora **justifica la complejidad adicional** del modelo.

Nota

Un RF simple que casi iguala a un GBM afinado suele ser preferible por gobernanza y mantenimiento.

Síntesis y Q&A

Mapa comparativo de modelos basados en árboles

Modelo	Sesgo	Varianza	Costo	Interpret.	Hp sensibles
Árbol único	Alto	Alta	Bajo	Alta	profundidad, poda
Random Forest	$\approx$ árbol	Baja	Medio	Media	m , n. árboles
Boosting (GBM)	Bajo	Baja c/reg.	Alto	Media	ν , prof., n. árboles

Regla práctica: empezar por Random Forest; pasar a boosting si se persigue el máximo desempeño.

- Los árboles capturan no-linealidades e interacciones, pero son inestables
- Random Forest reduce **varianza**: muchos árboles distintos promediados
- OOB $\approx$ validación cruzada casi gratis
- Boosting reduce **sesgo**: árboles que corrigen errores en cadena
- XGBoost: la implementación de boosting más usada, con regularización
- El **flujo** (partición, CV, tuning, test una vez) y una buena métrica importan tanto como el modelo

Módulo 3: SVM, k-NN y Naive Bayes

¿Preguntas?

-  Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR* 13.
-  Breiman, L. (1996). Bagging predictors. *Machine Learning* 24(2).
-  Breiman, L. (2001). Random forests. *Machine Learning* 45(1).
-  Breiman, L., Friedman, J.H., Olshen, R.A., & Stone, C.J. (1984). *Classification and Regression Trees*. Wadsworth.
-  Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD 2016*.

-  Friedman, J.H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29(5).
-  Grinsztajn, L. et al. (2022). Why do tree-based models still outperform deep learning on tabular data? *NeurIPS D&B*.
-  Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
-  Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS 2017*.
-  Prokhorenkova, L. et al. (2018). CatBoost. *NeurIPS 2018*.

-  Strobl, C. et al. (2007). Bias in random forest variable importance measures. *BMC Bioinformatics* 8.